
PORTFOLIO - GRADUAAT PROGRAMMEREN

Persoonlijk Dashboard

Een webapplicatie met vier API-gedreven widgets

De programmeur · Doel 3: Je wisselt gegevens gestructureerd uit

Auteur Jonas Trap

Opleiding Graduaat Programmeren, Thomas More Geel

Academiejaar 2025-2026

Type bewijs eigen project

Het project

Dit dashboard is een persoonlijk project: een single-page webapplicatie die vier dagelijkse noden samenbrengt op één centrale pagina. De vier widgets tonen het actuele weerbericht voor opgeslagen locaties, een dagelijks avondmaalrecept met filters voor dieet en bereidingstijd, de NASA-foto van de dag met uitleg, en een eenvoudig takenbeheer.

De kern van het project is communicatie met externe diensten: elke widget haalt zijn gegevens op bij een API en verwerkt die. Daarnaast worden eigen gegevens (taken en opgeslagen locaties) lokaal bewaard. Daardoor leent het project zich uitstekend om aan te tonen dat ik gegevens gestructureerd uitwissel.

De API's en gegevensstromen

- **OpenWeatherMap**: huidig weer en een 5-daagse voorspelling.
- **Spoonacular**: recepten met voedingswaarde en prijs.
- **NASA APOD**: de astronomische foto of video van de dag.
- **Vertaal-API**: de Engelse inhoud naar het Nederlands vertalen.
- **localStorage**: eigen opslag voor taken en opgeslagen locaties.

Broncode

De volledige broncode van dit project staat publiek op GitHub, zodat ze altijd terug te vinden is: github.com/jonastapmore/jonastap-projects/tree/main/dashboard

Gegevens gestructureerd uitwisselen

Ik koppel dit project aan dit doel omdat het volledig draait rond het uitwisselen van gestructureerde gegevens: met externe diensten en binnen de applicatie zelf. Hieronder werk ik de drie kernpunten van het doel uit.

1. App2App-communicatie met JSON

- Alle communicatie met externe diensten (OpenWeatherMap, Spoonacular, NASA, de vertaal-API) verloopt via **JSON**, het gestructureerde formaat voor App2App-communicatie.
- Aanvragen met de fetch-API en nette parameters: URLSearchParams bij Spoonacular (aantal, keuken, filters), querystrings bij OpenWeatherMap (q, units, lang), telkens met statuscontrole (response.ok).
- De antwoorden worden ingelezen met response.json() en getypeerd via interfaces (WeatherResponse, ForecastResponse, Recipe, ApodResponse), zodat de structuur expliciet vastligt en geneste data veilig te verwerken is (data.weather[0], nutrition.nutrients.find(...), list.filter(...)).
- Voor de eigen opslag wordt diezelfde JSON-structuur bewaard met JSON.stringify en weer ingelezen met JSON.parse.

2. Design patterns

- **Observer-patroon:** de PersistenceProvider laat widgets zich abonneren (addObserver) en verwittigt ze bij elke wijziging (notifyObservers), zodat de weergave automatisch herrendert.
- **Gelaagde structuur in de geest van MVC:** aparte lagen voor models (de data), components (de weergave/widgets) en data (de data laag met de providers).
- **Abstractie en uitwisselbaarheid:** een abstracte basisklasse PersistenceProvider met twee implementaties (localStorage en memory), en gedeelde basisklassen CustomElement en Page.
- **Factory-achtige methoden:** #createTaskItem(), #createLocationChip() en #createForecastDay() fabriceren telkens een DOM-element op basis van een data-item.

3. CRUD-operaties

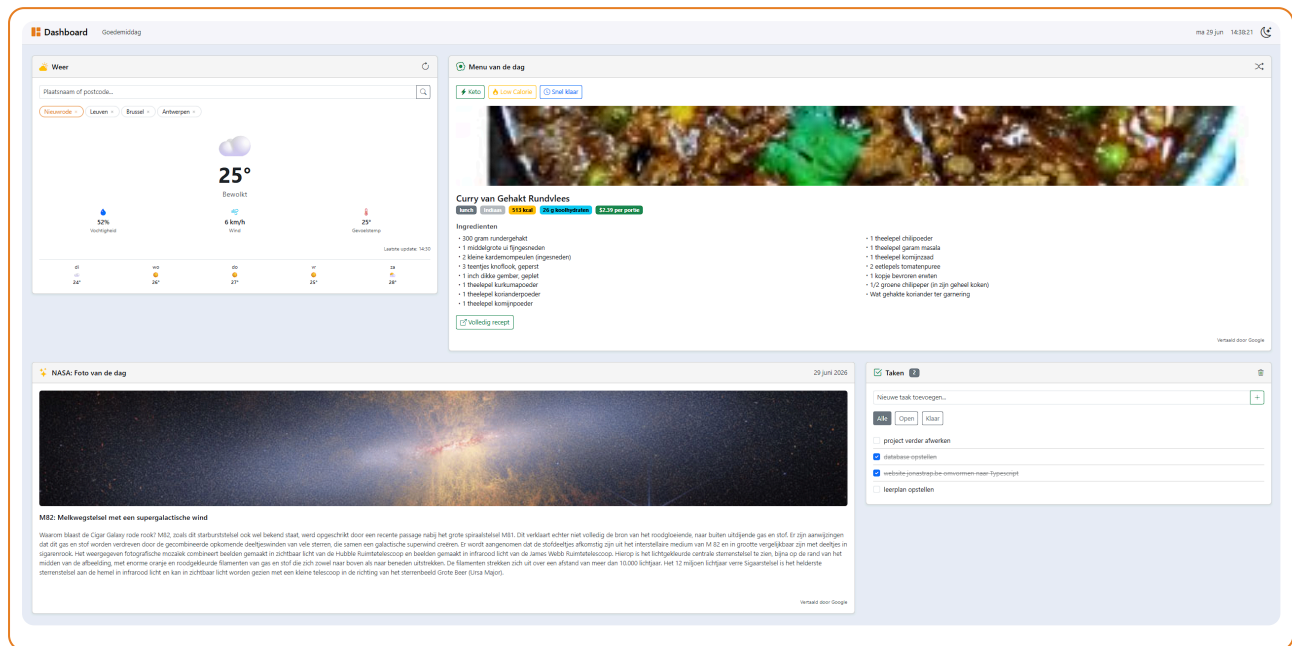
De PersistenceProvider voorziet de volledige CRUD-set, gebruikt door de taken- en weerwidget:

- **Create:** create() voegt een nieuwe taak of opgeslagen locatie toe.
- **Read:** getAll() en get() halen de gegevens op; de observer levert de actuele lijst.
- **Update:** update() wijzigt een taak, bijvoorbeeld afvinken als voltooid.
- **Delete:** delete() verwijdert een taak of locatie, en wist voltooide taken in één keer.

Betrouwbaar uitwisselen

- Foutafhandeling bij elke uitwisseling: controle van de status, nette meldingen, een automatische herhaling bij een tijdelijke serverfout, en een fallback voor de vertaling.

Het resultaat



Afbeelding 1: de widgets tonen gegevens die uit verschillende API's worden opgehaald en verwerkt (weer, recept met voedingswaarde, NASA-foto van de dag).

Een getypeerd datamodel

Voor elk antwoord leg ik de structuur vast in een interface, zodat de uitgewisselde gegevens voorspelbaar zijn:

```
// Vorm van een recept uit de Spoonacular-API (alleen de velden die we gebruiken)
interface Recipe {
  title: string;
  image: string;
  extendedIngredients: { original: string }[];
  pricePerServing: number; // in dollarcenten per portie
  nutrition: {
    nutrients: { name: string; amount: number; unit: string }[];
  };
}
```

Codefragment uit src/components/recipeWidget/recipeWidget.ts.

App2App-communicatie via JSON

De parameters worden netjes opgebouwd, de status gecontroleerd en het JSON-antwoord getypeerd ingelezen:

```
const params = new URLSearchParams({
  apiKey,
  number: '5',
  cuisine: keuken,
  sort: 'random',
  addRecipeNutrition: 'true',
});

const res = await fetch(`https://api.spoonacular.com/recipes/complexSearch?${params}`);
if (!res.ok) {
  throw new Error(`Geen recept gevonden (status ${res.status})`);
}
const data: { results: Recipe[] } = await res.json();
```

Gestructureerde uitwisseling met een externe API.

Design patterns en CRUD in de provider

De abstracte provider bundelt het observer-patroon en de volledige CRUD-set in één klasse:

```
export abstract class PersistenceProvider<T extends Persistable> {
  // Observer-patroon: widgets abonneren zich en worden bij elke wijziging verwittigd
  addObserver(observer: ChangeObserver<T>): Unsubscribe { /* ... */ }
  protected notifyObservers(data: T[]) { /* ... */ }

  // CRUD-operaties
  abstract create(data: Omit<T, 'id'>): Promise<T>; // Create
  abstract get(id: string): Promise<T>; // Read (één item)
```

```
    abstract getAll(): Promise<T[]>;                // Read (alle items)
    abstract update(id: string, data: T): Promise<T>; // Update
    abstract delete(id: string): Promise<void>;      // Delete
}
```

Codefragment uit `src/data/persistenceProvider.ts`: het observer-patroon en de CRUD-operaties. De concrete `LocalStoragePersistenceProvider` bewaart alles als JSON met `JSON.stringify`.

Zelfreflectie

Wat ging goed

Door voor elk API-antwoord een eigen interface te maken, werd het uitwisselen van gegevens overzichtelijk en veilig. De getypeerde modellen gaven me autoaanvulling en voorkwamen fouten: ik wist precies welke velden beschikbaar waren. Het opslaan en inlezen van eigen gegevens als JSON via een herbruikbare provider werkte voor zowel de taken als de opgeslagen locaties zonder dubbele code.

Wat ik geleerd heb

Ik leerde hoe belangrijk het is om de exacte structuur van een API-antwoord te kennen. Kleine verschillen in veldnamen veroorzaakten fouten: bij OpenWeatherMap heette een veld `feels_like` (niet `feels_Like`), bij de voorspelling was het `dt_txt` (niet `dt_text`), en Spoonacular gebruikt heel andere namen dan TheMealDB. Door de antwoorden te typeren en in de console te inspecteren, vond ik die verschillen snel. Ook leerde ik het onderscheid tussen **tijdelijke uitwisseling** (API-aanvragen) en **persistente opslag** (localStorage).

Uitdagingen

Externe diensten zijn niet altijd betrouwbaar of consistent. NASA gaf soms een tijdelijke serverfout, Spoonacular een limietfout, en TheMealDB gaf bij lege resultaten `null` in plaats van een lege lijst. Ik moest mijn verwerking daarop afstemmen met statuscontroles, een null-check en een automatische herhaling.

Wat beter kan

- Een echte **database** in plaats van `localStorage` voor robuustere opslag.
- **Schemavalidatie** van de API-antwoorden (bijvoorbeeld met een bibliotheek als Zod), zodat onverwachte structuren netjes opgevangen worden.

Conclusie

Dit project toont de drie kernpunten van het doel aan: **App2App-communicatie via JSON** met meerdere externe API's, het gebruik van **design patterns** (observer, abstractie, factory-achtige methoden en een gelaagde MVC-structuur), en volledige **CRUD-operaties** via een herbruikbare provider, telkens met de nodige controles en foutafhandeling.